



Laboratorio No. 1 Semana #13 – APIS con swagger & postman

Nombre: Martin Andrés Morales Mazariegos **Carné:** 1217025

Objetivo: Comprender y consumir una API REST mediante Swagger UI y Postman, identificando endpoints, métodos HTTP, parámetros, cuerpos JSON, códigos de respuesta y operaciones CRUD. El estudiante deberá interpretar la documentación de una API, ejecutar solicitudes y resolver un reto práctico utilizando evidencia técnica.

Investigación previa:

1. ¿Qué es una API REST?
Una API REST es un estilo de arquitectura de software para interconectar aplicaciones a través del protocolo HTTP. Utiliza los métodos estándar de la web para realizar operaciones sobre recursos de manera ligera, escalable y sin guardar estado de la sesión.
2. ¿Qué es un endpoint?
Un endpoint es la dirección específica (URI) donde una API expone un recurso determinado para que los clientes interactúen con él. Representa el punto exacto al que se envía una solicitud HTTP para realizar una acción.
3. ¿Qué diferencia existe entre una URL y un endpoint?
URL (Uniform Resource Locator): Es la dirección web completa utilizada para ubicar cualquier recurso en la red.
Endpoint: Es la combinación de la URL base con una ruta específica y un método HTTP que representa una funcionalidad o recurso concreto.
4. ¿Qué función cumplen los métodos HTTP GET, POST, PUT/PATCH y DELETE?
 - **GET:** Solicita o recupera datos de un recurso existente sin modificarlo.
 - **POST:** Envía información al servidor para crear un nuevo recurso.**PUT / PATCH:**
 - **PUT:** Reemplaza completamente un recurso existente con los nuevos datos enviados.
 - **PATCH:** Modifica o actualiza parcialmente un recurso (solo los campos enviados).
 - **DELETE:** Elimina un recurso específico del servidor.
5. ¿Qué es JSON y por qué es común en las APIs?
JSON (JavaScript Object Notation) es un formato ligero de intercambio de datos estructurado en pares de clave-valor ({"nombre": "Ana", "edad": 30}). Es el formato dominante en las APIs REST porque resulta fácilmente legible tanto para humanos como para máquinas, es agnóstico al lenguaje de programación, contando con soporte nativo en todos los lenguajes modernos y requiere menos ancho de banda que alternativas más pesadas como XML.
6. ¿Qué es un código de estado HTTP? Mencione ejemplos de 200, 404 y 500.
 - **200 OK:** La solicitud fue exitosa y el servidor devuelve la respuesta esperada.
 - **404 Not Found:** El recurso solicitado o el endpoint no existe en el servidor (error del cliente).
 - **500 Internal Server Error:** El servidor encontró un problema inesperado o un fallo en el código al procesar la solicitud (error del servidor).



7. ¿Qué información se puede encontrar en la respuesta de una API?
Una respuesta típica de una API incluye un código de estado HTTP que indica el éxito o fallo de la petición, encabezados con metadatos sobre el servidor y la seguridad, y un cuerpo que contiene los datos solicitados en formato JSON o los detalles de un error.
8. ¿Qué ventajas tiene utilizar Postman para probar una API antes de programarla?
Probar una API con Postman antes de escribir código permite validar el diseño mediante respuestas simuladas, facilita acordar el contrato de datos con el equipo de frontend o consumidores externos y ahorra tiempo al detectar problemas de arquitectura de manera temprana.
9. ¿Qué es y para qué sirve Swagger UI?
Swagger UI es una herramienta que genera documentación visual e interactiva a partir de una especificación OpenAPI. Sirve para que desarrolladores y clientes puedan visualizar los endpoints disponibles, ver los parámetros y estructuras esperadas, e incluso probar las peticiones directamente desde el navegador web sin instalar software adicional.
10. ¿Qué función cumple Postman en el desarrollo y prueba de APIs?
Postman actúa como un cliente HTTP completo y entorno de pruebas que permite construir y enviar solicitudes personalizadas, inspeccionar respuestas detalladas como latencias y códigos de estado, organizar peticiones en colecciones reusables y automatizar pruebas unitarias y de integración sobre los endpoints.

1. Preparación del entorno

Instalar o utilizar Postman: <https://www.postman.com/downloads/>

API a explorar, Swagger Petstore 3: <https://petstore3.swagger.io/>

Importante: La actividad se realizará mediante solicitudes HTTP GET y análisis de las respuestas JSON. No se requiere API Key para esta práctica.

2. Exploración inicial

Ingresa a Swagger Petstore y localice las operaciones relacionadas con /pet

- Identifique un endpoint que permita consultar una mascota por ID.
- Indique el método HTTP utilizado.
- Indique qué parámetro necesita la solicitud.
- Ejecute la solicitud con Try it out y registre el código HTTP recibido.
- Explique con sus palabras qué información contiene la respuesta.

Evidencia: captura de Swagger mostrando la operación y su respuesta.



2.1.

The screenshot shows the Swagger UI for the petstore3 API. The endpoint `GET /pet/{petId}` is selected, with the description "Find pet by ID." and "Returns a single pet." The parameter `petId` is defined as a required integer (path). The response is a 200 OK status with a successful operation. The example JSON response is:

```
{
  "id": 10,
  "name": "doggie",
  "category": {
    "id": 1,
    "name": "Dogs"
  },
  "photoUrls": [
    "string"
  ],
  "tags": [
    "tag"
  ]
}
```

On the right, the Chrome DevTools Network tab shows the request details for `openapi.json`. The request method is GET, and the status code is 200 OK. The response headers include `Content-Type: application/json` and `Date: Wed, 30 Sep 2026 13:36:39 GMT`.

2.2. GET

2.3. getId

3. Crear un recurso con POST

Utilice POST `/pet` para crear una mascota. Construya un JSON propio; no copie exactamente el ejemplo de Swagger.

El JSON debe incluir, como mínimo:

- id
- name
- status
- category
- tags

Registre el ID utilizado y verifique la respuesta. Luego consulte nuevamente la mascota por medio de GET `/pet/{petId}`.

Evidencia: JSON enviado + respuesta de creación + consulta posterior.



POST

/pet Add a new pet to the store.

Add a new pet to the store.

Parameters

Cancel

Reset

No parameters

Request body required

application/json

Create a new pet in the store

Edit Value

Schema

```
{  "id": 12170,  "name": "doggie",  "category": {    "id": 1,    "name": "Dogs"  },  "photoUrls": [    "string"  ],  "tags": [    {      "id": 0,      "name": "string"    }  ],  "status": "available"}
```

Execute

Clear

Responses

Curl

```
curl -X 'POST' \  https://petstore.swagger.io/v3/pet \  -H 'accept: application/json' \  -H 'Content-Type: application/json' \  -d '{  "id": 12170,  "name": "doggie",  "category": {    "id": 1,    "name": "Dogs"  },  "photoUrls": [    "string"  ],  "tags": [    {      "id": 0,      "name": "string"    }  ],  "status": "available"}
```

https://petstore.swagger.io/v3/pet

Server response

Code

Details

200

Response body

```
{  "id": 12170,  "category": {    "id": 1,    "name": "Dogs"  },  "name": "doggie",  "photoUrls": [    "string"  ],  "tags": [    {      "id": 0,      "name": "string"    }  ],  "status": "available"}
```

Download

Response headers

```
access-control-allow-headers: Content-Type,api_key,Authorization
access-control-allow-methods: GET,POST,DELETE,PUT
access-control-allow-origin: *
access-control-expose-headers: Content-Disposition
content-length: 140
content-type: application/json
date: Sat,03 Oct 2026 02:43:24 GMT
server: Jetty(9.4.52.v20231009)
```

Responses

Code

Description

Links

200

Successful operation

No links

Media type

application/json

Controls

Accept header

Example Value

Schema

```
{  "id": 10,  "name": "doggie",  "category": {    "id": 1,    "name": "Dogs"  },  "photoUrls": [    "string"  ],  "tags": [    {      "id": 0,      "name": "string"    }  ],  "status": "available"}
```

400

Invalid input

No links

422

Validation exception

No links

default

Unexpected error

No links



4. Modificar el recurso

- Modifique la mascota creada utilizando PUT /pet. Cambie al menos dos atributos, por ejemplo nombre y estado.
- ¿Qué método HTTP utilizó y por qué?
Se utilizó el método **PUT** porque está diseñado para actualizar un recurso existente en el servidor enviando sus datos completos en el cuerpo de la petición.
- ¿Qué datos fueron enviados en el body?

```
{  
  "id": 12170,  
  "name": "doggie",  
  "category": {  
    "id": 1, "name": "Dogs"  
  }, "photoUrls": [  
    "string"  
  ],  
  "tags": [  
    {  
      "id": 0,  
      "name": "string"  
    }  
  ],  
  "status": "available"  
}
```

- ¿Qué código HTTP devolvió la API?
La API devolvió el código 200 (OK / Successful operation) porque la petición realizada mediante el método GET a la URL encontró exitosamente la mascota con el ID 12170 registrada en el servidor. Este código confirma que la consulta fue correcta y, en respuesta, el servidor entregó el objeto JSON completo con la información solicitada.
- ¿Cómo comprobó que el cambio realmente ocurrió?
Se comprobó realizando una petición de tipo GET enviando el id 12170 en la URL. Al recibir una respuesta con código 200 OK y la información actualizada en el cuerpo del JSON, se confirmó que el registro de la mascota existía y conservaba los datos asignados.



pet Everything about your Pets [Find out more](#)

PUT /pet Update an existing pet.

Update an existing pet by Id.

Parameters

No parameters

Request body required

application/json

Update an existent pet in the store

Edit Value Schema

```
{
  "id": 12170,
  "name": "doggie",
  "category": {
    "id": 1,
    "name": "Dogs"
  },
  "photoUrls": [
    "string"
  ],
  "tags": [
    {
      "id": 0,
      "name": "string"
    }
  ],
  "status": "available"
}
```

Execute Clear

Responses

Curl

```
curl -X 'PUT' \
  'https://petstore3.swagger.io/api/v3/pet' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "id": 12170,
    "name": "doggie",
    "category": {
      "id": 1,
      "name": "Dogs"
    },
    "photoUrls": [
      "string"
    ],
    "tags": [
      {
        "id": 0,
        "name": "string"
      }
    ],
    "status": "available"
  }'
```

Request URL

https://petstore3.swagger.io/api/v3/pet

Server response

Code Details

200

Response body

```
{
  "id": 12170,
  "category": {
    "id": 1,
    "name": "Dogs"
  },
  "name": "doggie",
  "photoUrls": [
    "string"
  ],
  "tags": [
    {
      "id": 0,
      "name": "string"
    }
  ],
  "status": "available"
}
```

Download

Response headers

```
access-control-allow-headers: Content-Type,api_key,Authorization
access-control-allow-methods: GET,POST,DELETE,PUT
access-control-allow-origin: *
access-control-expose-headers: Content-Disposition
content-length: 340
content-type: application/json
date: Sat, 01 Oct 2026 02:44:20 GMT
server: Jetty/9.4.53.v20211009
```

Responses

Code	Description	Links
200	Successful operation	No links
400	Invalid ID supplied	No links
404	Pet not found	No links
422	Validation exception	No links
default	Unexpected error	No links



5. Consultar utilizando parámetros

- Utilice GET /pet/findByStatus y consulte las mascotas con estado available.
- Identifique el parámetro utilizado.
- Explique dónde se envía ese parámetro.
- Observe la estructura de la respuesta.
- Indique qué sucede si cambia el valor del parámetro por pending.

The screenshot shows the Swagger UI interface for a pet store API. The top section displays the endpoint `petid` (required, integer) with a description "ID of pet to return". A text input field contains the value `12170`. Below the input are "Execute" and "Clear" buttons. The "Responses" section shows the result of the request. It includes a curl command: `curl -X 'GET' \ 'https://petstore3.swagger.io/api/v3/pet/12170' \ -H 'accept: application/json'`. The "Request URL" is `https://petstore3.swagger.io/api/v3/pet/12170`. The "Server response" shows a status code of 200. The "Response body" is a JSON object:

```
{  "id": 12170,  "category": {    "id": 1,    "name": "Dogs"  },  "name": "doggie",  "photoUrls": [    "string"  ],  "tags": [    {      "id": 0,      "name": "string"    }  ],  "status": "available"}
```

. The "Response headers" section lists various headers like `access-control-allow-headers`, `access-control-allow-methods`, `access-control-allow-origin`, `access-control-expose-headers`, `content-length`, `content-type`, `date`, and `server`. At the bottom, a table shows the response details: status code 200, description "successful operation", and media type "application/json".

6. Reproducir las operaciones en Postman

- Ahora reproduzca en Postman las operaciones realizadas en Swagger. Cree una colección llamada 'MIA - Semana 13 - API'.
- La colección debe contener, como mínimo:
- GET – Consultar mascota por ID
- POST – Crear mascota
- PUT – Modificar mascota
- GET – Buscar por status
- DELETE – Eliminar mascota
- En cada solicitud configure manualmente método, URL, parámetros, headers y body cuando corresponda. No utilice únicamente la opción de importar una colección.



POST:

Postman interface showing a POST request to `https://petstore3.swagger.io/api/v3/pet`. The request body is a JSON object:

```
{
  "id": 12179,
  "name": "Ruggie",
  "category": {
    "id": 1,
    "name": "dogs"
  },
  "photoUrls": [
    "string"
  ],
  "tags": [
    {
      "id": 0,
      "name": "string"
    }
  ],
  "status": "available"
}
```

The response is a 200 OK status with a 15 ms response time. The response body is a JSON object:

```
{
  "id": 12179,
  "name": "Ruggie",
  "category": {
    "id": 1,
    "name": "dogs"
  },
  "photoUrls": [
    "string"
  ],
  "tags": [
    {
      "id": 0,
      "name": "string"
    }
  ],
  "status": "available"
}
```

PUT:

Postman interface showing a PUT request to `https://petstore3.swagger.io/api/v3/pet`. The request body is a JSON object:

```
{
  "id": 12179,
  "name": "Ruggie_updated",
  "category": {
    "id": 1,
    "name": "dogs"
  },
  "photoUrls": [
    "string"
  ],
  "tags": [
    {
      "id": 0,
      "name": "string"
    }
  ],
  "status": "available"
}
```

The response is a 200 OK status with a 43 ms response time. The response body is a JSON object:

```
{
  "id": 12179,
  "name": "Ruggie_updated",
  "category": {
    "id": 1,
    "name": "dogs"
  },
  "photoUrls": [
    "string"
  ],
  "tags": [
    {
      "id": 0,
      "name": "string"
    }
  ],
  "status": "available"
}
```



GET:

Postman interface showing a GET request to `https://petstore3.swagger.io/api/v3/pet/12170`. The response is a 200 OK status with a JSON body:

```
{
  "id": 12170,
  "category": {
    "id": 1,
    "name": "dogs"
  },
  "name": "doggie updated",
  "photoUrls": [
    "string"
  ],
  "tags": [
    {
      "id": 0,
      "name": "string"
    }
  ],
  "status": "available"
}
```

DELETE:

Postman interface showing a DELETE request to `https://petstore3.swagger.io/api/v3/pet/12170`. The response is a 200 OK status with a JSON body:

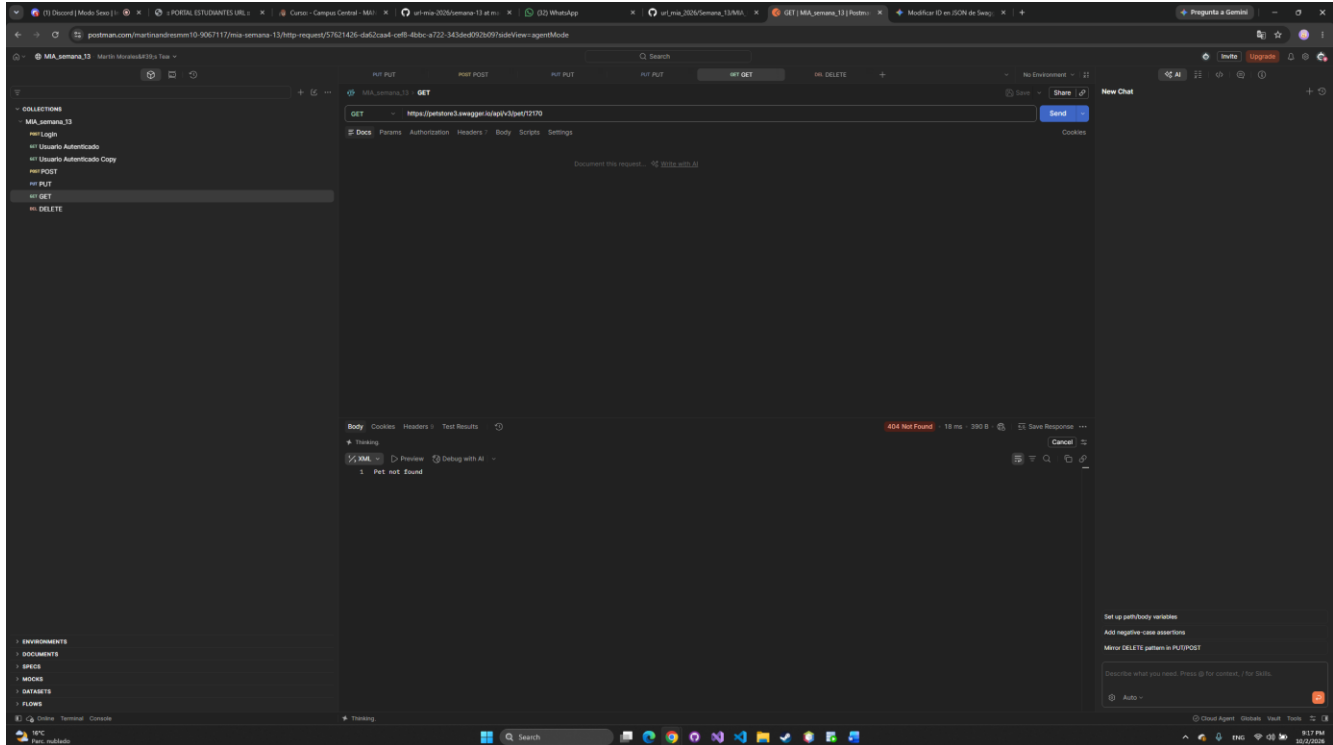
```
{
  "Pet deleted"
}
```



Una API también debe analizarse cuando algo sale mal. Realice al menos tres pruebas que generen respuestas diferentes a una solicitud exitosa.

Ejemplos de escenarios:

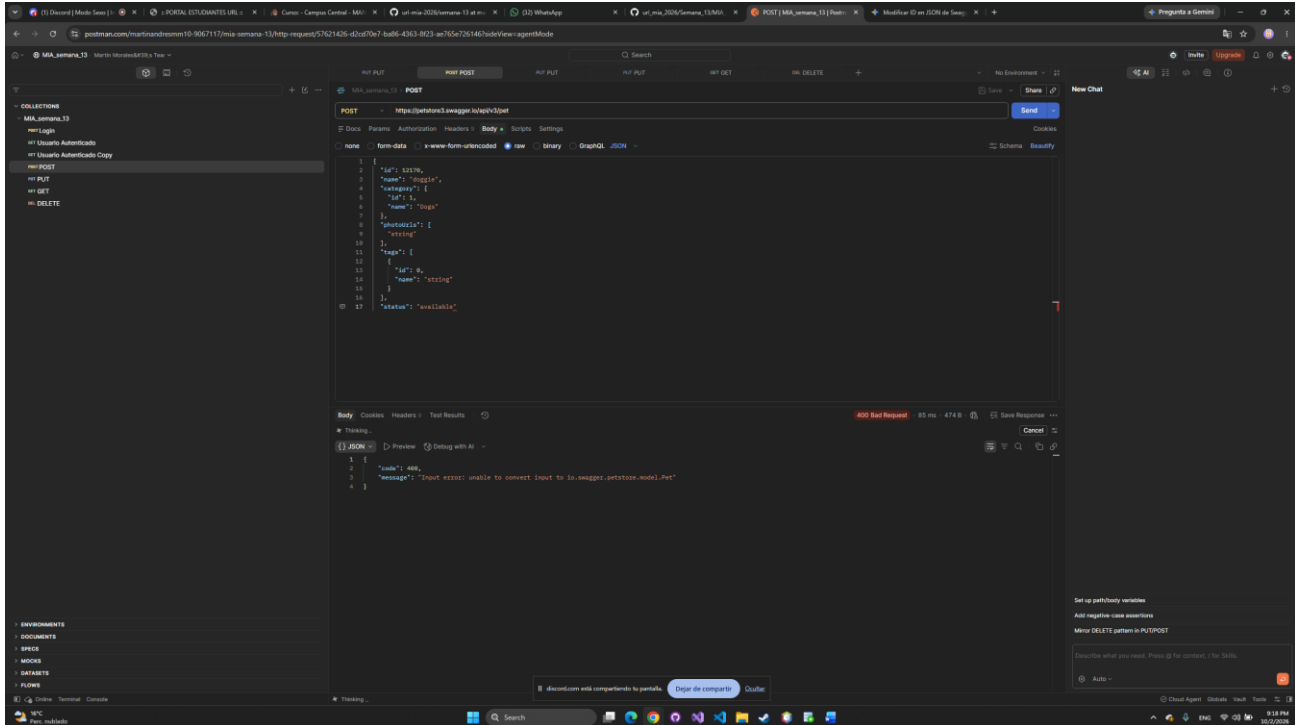
1. Consultar un ID inexistente.



- Solicitud realizada.
GET
- Código HTTP recibido.
404 Not Found
- Respuesta de la API.
Pet not found
- Qué esperaba que ocurriera.
Se esperaba que el servidor no encontrara el recurso y devolviera un código de estado de error de cliente `404 Not Found` con un mensaje indicando que la mascota no existe.
- Qué ocurrió realmente.
El servidor procesó la consulta y devolvió exitosamente el código HTTP `404 Not Found` junto con el mensaje `Pet not found` en el cuerpo de la respuesta.
- Explicación de la causa
Al realizar una petición `GET` a un endpoint de consulta por ID, el servidor busca el identificador en la base de datos. Dado que el ID `12170` no se encuentra registrado en el sistema, la API responde con un código de estado 404, que es el estándar HTTP para notificar que el recurso solicitado no fue encontrado.



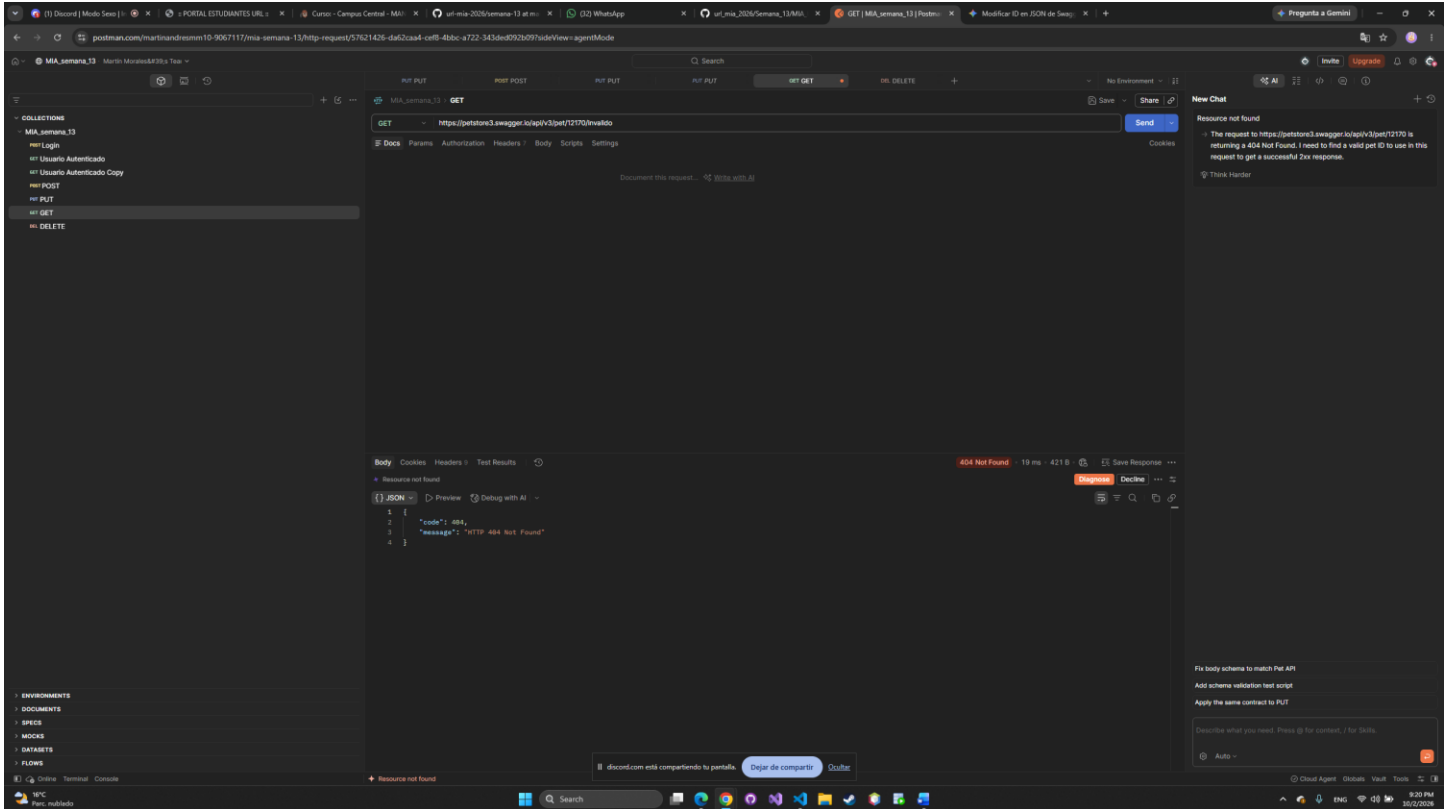
2. Enviar un JSON incorrecto o incompleto.



- Solicitud realizada.
POST
- Código HTTP recibido.
400 Bad Request
- Respuesta de la API.
{ "code": 400, "message": "Input error: unable to convert input to io.swagger.petstore.model.Pet" }
- Qué esperaba que ocurriera.
Se esperaba que la API rechazara la solicitud devolviendo un código de error de cliente `400 Bad Request`, indicando que la sintaxis de la estructura del JSON era inválida o no se podía procesar.
- Qué ocurrió realmente.
El servidor retornó el código `400 Bad Request` con un mensaje indicando que no pudo convertir la entrada enviada al modelo esperado (`unable to convert input to io.swagger.petstore.model.Pet`).
- Explicación de la causa
Al enviar una cadena JSON con un error de sintaxis (como la falta del delimitador de cierre `}`), el parser del servidor no puede deserializar el contenido de la petición para transformarlo en un objeto del tipo de modelo requerido. Por ello, la API detiene el procesamiento inmediatamente y responde con un error estándar 400, señalando que los datos suministrados por el cliente tienen una estructura incorrecta.



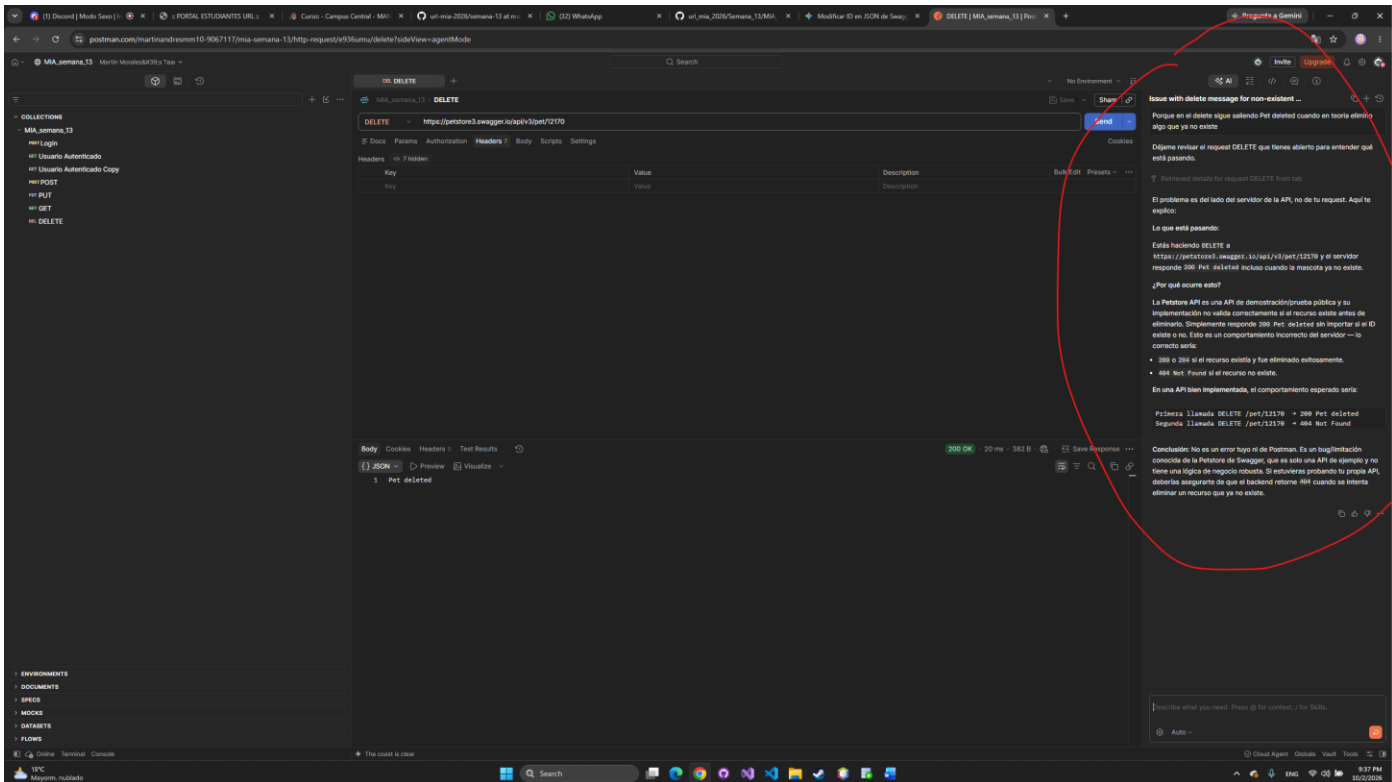
3. Utilizar un parámetro con un valor no esperado.



- Solicitud realizada.
GET
- Código HTTP recibido.
404 Not Found
- Respuesta de la API.
{ "code": 404, "message": "HTTP 404 Not Found" }
- Qué esperaba que ocurriera.
Se esperaba que la API rechazara la solicitud o devolviera un error indicando que la ruta o el parámetro ingresado no es válido para ese endpoint.
- Qué ocurrió realmente.
El servidor retornó el código `404 Not Found` especificando en el cuerpo del JSON que la ruta/recurso consultado no existe (`"HTTP 404 Not Found"`).
- Explicación de la causa
Al concatenar un valor no esperado o un segmento adicional de ruta al final de la URL del endpoint, la petición se dirige a un path que no está mapeado en la API. Como el servidor web/framework no encuentra ninguna ruta registrada que coincida con `/api/v3/pet/12170/invalido`, responde de forma predeterminada con un estado 404 Not Found.



4. Intentar eliminar un recurso que ya no existe.



- a) Solicitud realizada.
DELETE
- b) Código HTTP recibido.
200 OK
- c) Respuesta de la API.
PET deleted
- d) Qué esperaba que ocurriera.
- e) En un diseño REST estándar, al intentar eliminar un recurso que ya fue eliminado o que no existe, se esperaba recibir un código de error `404 Not Found`.
- f) Qué ocurrió realmente.
El servidor retornó un código de estado `200 OK` con el mensaje `""Pet deleted""`, confirmando la eliminación a pesar de que el recurso ya no existía en el Sistema.
- g) Explicación de la causa
Esto ocurre por una limitación propia en la implementación pública de la API de prueba Swagger Petstore. La API no valida previamente si el ID solicitado existe en la base de datos antes de procesar el borrado, respondiendo de forma predeterminada con un mensaje de éxito `200 Pet deleted` sin importar si el recurso estaba o no registrado.

Para cada prueba registre:

- h) Solicitud realizada.
- i) Código HTTP recibido.
- j) Respuesta de la API.
- k) Qué esperaba que ocurriera.
- l) Qué ocurrió realmente.
- m) Explicación de la causa



Suponga que usted recibe el siguiente requerimiento de un sistema:

“El sistema debe registrar una mascota, consultar su información, actualizar su estado y finalmente eliminarla. Si el recurso no existe, el sistema debe poder identificarlo mediante la respuesta de la API.”

Resuelva el reto:

- Defina qué endpoint utilizaría para cada operación.
- Defina el método HTTP correspondiente.
- Indique qué respuesta debe analizar el programa.
- Ejecute el flujo completo en Postman.

Capture evidencia de cada paso.

The screenshot shows the Postman interface with a DELETE request configured. The URL is `https://petstore3.swagger.io/api/v3/pet`. The body is a JSON object: `{ "id": 12378, "name": "doggie", "photoUrls": [], "tags": [], "status": "available" }`. The response status is `200 OK`. The right sidebar displays a message: "Issue with delete message for non-existent ...". The message explains that the problem is the ID of the resource being deleted, which does not exist in the server. It states that the Petstore API is a public demo API and that the correct behavior is to return `200 OK` when deleting a resource that exists and `404 Not Found` when it does not exist. The sidebar also shows a list of requests: "Primera llamada DELETE /pet/12378" (200 OK) and "Segunda llamada DELETE /pet/12378" (404 Not Found).



The screenshot displays the Postman application interface. The main workspace shows a GET request to `https://petstore3.swagger.io/api/v3/pet/12170`. The response is a 200 OK with a JSON body:

```
{
  "id": 12170,
  "name": "raggy",
  "photoUrls": [],
  "tags": [],
  "status": "available"
}
```

The right sidebar contains an "Issue with delete message for non-existent ..." section. It explains that the problem is on the server side, not the request. It states that the Petstore API is a demo API and its implementation is not correct. It mentions that the API should return 200 OK when the resource exists and 404 Not Found when it does not. The sidebar also includes a "Conclusion" section stating that the issue is not a bug in Postman but a bug in the API implementation.



The image displays two screenshots of the Postman application interface, showing the results of an API test for a DELETE endpoint.

Top Screenshot: The interface shows a DELETE request to `https://petstore3.swagger.io/api/v3/pet/12170`. The response is a 200 OK status, indicating the resource was successfully deleted. The response body is `{ "pet": "deleted" }`.

Bottom Screenshot: The interface shows a GET request to `https://petstore3.swagger.io/api/v3/pet/12170`. The response is a 404 Not Found status, indicating the resource no longer exists. The response body is `{ "pet": "not found" }`.

Entrega

Dentro del repositorio de GitHub, cree la carpeta semana-13/lab1/ y coloque en ella un PDF con la solución del laboratorio.

Incluya también los archivos de respaldo que generados durante la práctica, así como los pantallazos que evidencien las



Universidad
Rafael Landívar

Identidad Jesuita en Guatemala

pruebas realizadas.

Enviar link de github a plataforma de la universidad.

Facultad de Ingeniería

Facultad de Informática y sistemas

Manejo e Implementación de Archivos

Miércoles, 30 de septiembre de 2026